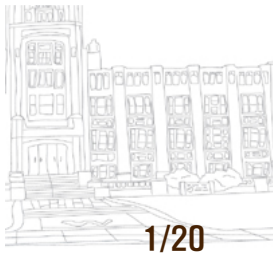
3장. 연산자

이것이 자바다



Contents

❖ 목차

- 1절. 연산자와 연산식
- 2절. 연산의 방향과 우선 순위
- 3절. 단항 연산자
- 4절. 이항 연산자
- 5절. 삼항 연산자(? :)



1절. 연산자와 연산식

❖ 연산이란?

- 데이터를 처리하여 결과를 산출하는 것

- 연산자(Operations)
  - 연산에 사용되는 표시나 기호(+, -, *, /, %, =, …)

- 피연산자(Operand): 연산 대상이 되는 데이터(리터럴, 변수)

- 연산식(Expressions)
  - 연산자와 피연산자를 이용하여 연산의 과정을 기술한 것



1절. 연산자와 연산식

❖ 연산자의 종류 (p.64)

연산자 종류	연산자	피연산자 수	산출값 타입	기능 설명
산술	+, -, *, /, %	이항	숫자	사칙연산 및 나머지 계산
부호	+, -	단항	숫자	음수와 양수의 부호
문자열	+	이항	문자열	두 문자열을 연결
대입	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=	이항	다양	우변의 값을 좌변의 변수에 대입
증감	++, --	단항	숫자	1 만큼 증가/감소
비교	==, !=, >, <, >=, <=, instanceof	이항	boolean	값의 비교
논리	!, &, |, &&, ||	단항 이항	boolean	논리적 NOT, AND, OR 연산
조건	(조건식) ? A : B	삼항	다양	조건식에 따라 A 또는 B 중 하나를 선택
비트	~, &, |, ^	단항 이항	숫자 blooean	비트 NOT, AND, OR, XOR 연산
쉬프트	>>, <<, >>>	이항	숫자	비트를 좌측/우측으로 밀어서 이동

2절. 연산의 방향과 우선 순위

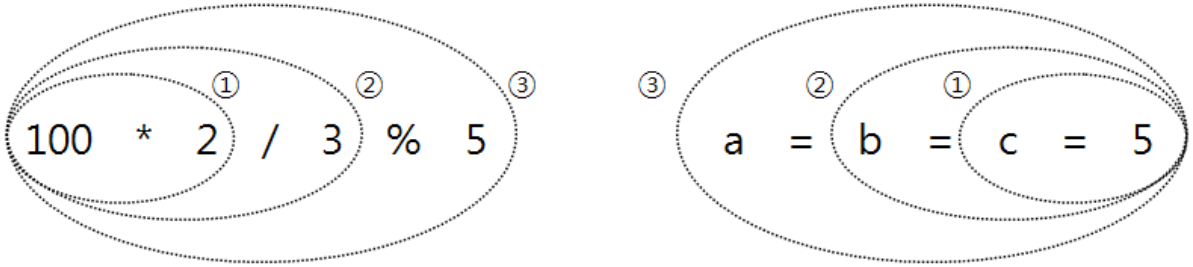
❖ 연산의 방향과 우선 순위

- **연산자의 우선 순위에 따라 연산된다.**

```
x > 0 && y < 0
```

- **동일한 우선 순위의 연산자는 연산의 방향 존재**

*, /, %는 같은 우선 순위를 갖고 있다. 이들 연산자는 연산 방향이 왼쪽에서 오른쪽으로 수행된다. 100 * 2 가 제일 먼저 연산되어 200 이 산출되고, 그 다음 200 / 3 이 연산되어 66 이 산출된다. 그 다음으로 66 % 5 가 연산되어 1 이 나온다.



하지만 단항 연산자(++, --, ~, !), 부호 연산자(+, -), 대입 연산자(=, +=, -=, ...)는 오른쪽에서 왼쪽(←)으로 연산된다. 예를 들어 다음 연산식을 보자.

2절. 연산의 방향과 우선 순위

❖ 연산의 방향과 우선 순위

연산자	연산 방향	우선 순위
증감(++, --), 부호(+, -), 비트(~), 논리(!)	←	높음
산술(*, /, %)	→	
산술(+, -)	→	
쉬프트(<<, >>, >>>)	→	
비교(<, >, <=, >=, instanceof)	→	
비교(==, !=)	→	
논리(&)	→	
논리(^)	→	
논리(|)	→	
논리(&&)	→	
논리(||)	→	
조건(?:)	→	
대입(=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=)	←	낮음

연산 방향과 우선순위 정리

- 단항, 이항, 삼항 연산자 순으로 우선순위를 가진다.
- 산술, 비교, 논리, 대입 연산자 순으로 우선순위를 가진다.
- 단항과 대입 연산자를 제외한 모든 연산의 방향은 왼쪽에서 오른쪽이다.
- 복잡한 연산식에는 괄호()를 사용해서 우선순위를 정해준다.



3절. 단항 연산자

❖ 단항연산자란?

- 피연산자가 1개인 연산자

❖ 단항 연산자의 종류 (p.68~)

- 부호 연산자: +, -
  - boolean 타입과 char 타입을 제외한 기본 타입에 사용 가능
  - 부호 연산자의 산출 타입은 int

```
short s=100;
short result = -s; //컴파일 에러
```

↓

int result3=-s로 변경

SignOperatorExample.java(69)_부호 연산자

```java
public class SignOperatorExample {
    public static void main(String[] args) {
        int x = -100;
        int result1 = +x;
        int result2 = -x;
        System.out.println("result1=" + result1);
        System.out.println("result2=" + result2);

        short s = 100;
        //short result3 = -s;  //컴파일 에러
        int result3 = -s;
        System.out.println("result3=" + result3);

    }
}
```

```
C:₩Windows₩system32₩cmd.exe
result1=-100
result2=100
result3=-100
계속하려면 아무 키나 누르십시오 . . .
```

3절. 단항 연산자

- 증감 연산자: ++, --
  - 변수의 값을 1증가 시키거나 (++) 1 감소 (--) 시키는 연산자
  - 증감 연산자가 변수 뒤에 있으면 다른 연산자 먼저 처리 후 증감 연산자 처리



IncreaseDecreaseOperatorExample.java(71)_증감 연산자

```java
public class IncreaseDecreaseOperatorExample {
    public static void main(String[] args) {
        int x = 10;
        int y = 10;
        int z;

        System.out.println("----------------------");
        x++;
        ++x;
        System.out.println("x=" + x);

        System.out.println("----------------------");
        y--;
        --y;
        System.out.println("y=" + y);

        System.out.println("----------------------");
        z = x++;
        System.out.println("z=" + z);
        System.out.println("x=" + x);

        System.out.println("----------------------");
        z = ++x;
        System.out.println("z=" + z);
        System.out.println("x=" + x);

        System.out.println("----------------------");
        z = ++x + y++;
        System.out.println("z=" + z);
        System.out.println("x=" + x);
        System.out.println("y=" + y);
    }
}
```



ArithmeticOperatorExample.java

```java
package ch03.sec02;

public class ArithmeticOperatorExample {
    public static void main(String[] args) {
        byte v1 = 10;
        byte v2 = 4;
        int v3 = 5;
        long v4 = 10L;

        int result1 = v1 + v2;          //모든 피연산자는 int 타입으로 자동 변환 후 연산
        System.out.println("result1: " + result1);

        long result2 = v1 + v2 - v4;    //모든 피연산자는 long 타입으로 자동 변환 후 연산
        System.out.println("result2: " + result2);

        double result3 = (double) v1 / v2;  //double 타입으로 강제 변환 후 연산
        System.out.println("result3: " + result3);

        int result4 = v1 % v2;
        System.out.println("result4: " + result4);
    }
}
```

3절. 단항 연산자

❖ 단항 연산자의 종류

▪ 논리 부정 연산자: !(72)

- **Boolean type 에만 사용가능**

연산식		설명
!	피연산자	피연산자가 true 이면 false 값을 산출 피연산자가 false 이면 true 값을 산출

```
1
2  public class DenyLogicOperatorExample {
3      public static void main(String[] args) {
4          boolean play = true;
5          System.out.println(play);
6
7          play = !play;
8          System.out.println(play);
9
10         play = !play;
11         System.out.println(play);
12         }
13 }
14
```

```
C:₩Windows₩system32₩cmd.exe
true
false
true
계속하려면 아무 키나 누르십시오 . . .
```

3절. 단항 연산자

- **비트 반전 연산자: ~**
  - byte, short, int, long 타입만 피연산자가 될 수 있다.
  - 비트값을 반전(0□1, 1□0)시킨다.

연산식		설명
~	10 (0 0 … 0 1 0 1 0)	산출결과: -11 (1 1 … 1 0 1 0 1)

BitReverseOperatorExample.java_비트 반전 연산자(75)

```
    ├---+----1----+----2----+----3----+----4----+----5----+----6----+----7----+---
 1  public class BitReverseOperatorExample {
 2      public static void main(String[] args) {
 3          int v1 = 10;
 4          int v2 = ~v1;
 5          int v3 = ~v1 + 1;
 6          System.out.println(toBinaryString(v1) + " (십진수: " + v1 + ")");
 7          System.out.println(toBinaryString(v2) + " (십진수: " + v2 + ")");
 8          System.out.println(toBinaryString(v3) + " (십진수: " + v3 + ")");
 9          System.out.println();
10
11          int v4 = -10;
12          int v5 = ~v4;
13          int v6 = ~v4 + 1;
14          System.out.println(toBinaryString(v4) + " (십진수: " + v4 + ")");
15          System.out.println(toBinaryString(v5) + " (십진수: " + v5 + ")");
16          System.out.println(toBinaryString(v6) + " (십진수: " + v6 + ")");
17      }
18
19      public static String toBinaryString(int value) {
20          String str = Integer.toBinaryString(value);
21          while(str.length() < 32) {
22              str = "0" + str;
23          }
24          return str;
25      }
26  }
27
```

```
C:₩Windows₩system32₩cmd.exe
00000000000000000000000000001010 (십진수: 10)
11111111111111111111111111110101 (십진수: -11)
11111111111111111111111111110110 (십진수: -10)

11111111111111111111111111110110 (십진수: -10)
00000000000000000000000000001001 (십진수: 9)
00000000000000000000000000001010 (십진수: 10)
계속하려면 아무 키나 누르십시오 . . .
```

4절. 이항 연산자

❖ 이항 연산자란? (p.75~)

- 피연산자가 2개인 연산자

- 종류
  - 산술 연산자: +, -, *, /, %
  - 문자열 연결 연산자: +
  - 대입 연산자: =, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=
  - 비교 연산자: <, <=, >, >=, ==, !=
  - 논리 연산자: &&, ||, &, |, ^, !
  - 비트 논리 연산자: &, |, ^
  - 비트 이동 연산자: <<, >>, >>>



4절. 이항 연산자

❖ 산술 연산자

- boolean 타입을 제외한 모든 기본 타입에 사용 가능
- 결과값 산출할 때 Overflow 주의
- 정확한 계산은 정수를 사용
- NaN과 Infinity 연산은 주의할 것

연산식			설명
피연산자	+	피연산자	덧셈 연산
피연산자	-	피연산자	뺄셈 연산
피연산자	*	피연산자	곱셈 연산
피연산자	/	피연산자	좌측 피연산자를 우측 피연산자로 나눗셈 연산
피연산자	%	피연산자	좌측 피연산자를 우측 피연산자로 나눈 나머지를 구하는 연산

```
int result = num % 3;
```

0, 1, 2 중의 한 값 — num 을 3 으로 나눈 나머지



ArithmeticOperatorExample.java_산술 연산자(78)

- boolean 타입을 제외한 모든 기본 타입에 사용 가능

```
1  package ch03.sec02;
2
3  public class ArithmeticOperatorExample {
4      public static void main(String[] args) {
5          byte v1 = 10;
6          byte v2 = 4;
7          int v3 = 5;
8          long v4 = 10L;
9
10         int result1 = v1 + v2;           //모든 피연산자는 int 타입으로 자동 변환 후 연산
11         System.out.println("result1: " + result1);
12
13         long result2 = v1 + v2 - v4;     //모든 피연산자는 long 타입으로 자동 변환 후 연산
14         System.out.println("result2: " + result2);
15
16         double result3 = (double) v1 / v2;  //double 타입으로 강제 변환 후 연산
17         System.out.println("result3: " + result3);
18
19         int result4 = v1 % v2;
20         System.out.println("result4: " + result4);
21     }
22 }
```

CharOperationExample.java_char type 연산(79)

- boolean 타입을 제외한 모든 기본 타입에 사용 가능_char 타입 연산

```
    ├---+---1---+---2---+---3---+---4---+-
 1  public class CharOperationExample {
 2      public static void main(String[] args) {
 3          char c1 = 'A' + 1;
 4          char c2 = 'A';
 5          //char c3 = c2 + 1;     //컴파일 에러
 6          System.out.println("c1: " + c1);
 7          System.out.println("c2: " + c2);
 8          //System.out.println("c3: " + c3);
 9      }
10  }
11
```

```
C:₩Windows₩system32₩cmd.exe
c1: B
c2: A
계속하려면 아무 키나 누르십시오 . . .
```

OverflowExample.java_오버플로우 (80)

- 결과값 산출할 때 Overflow 문제

```
    ├---+----1----+----2----+----3----+----4----+--
 1  public class OverflowExample {
 2      public static void main(String[] args) {
 3
 4          int x = 1000000;
 5          int y = 1000000;
 6          int z = x * y;
 7          System.out.println(z);
 8
 9      }
10  }
11
12
```

```
C:₩Windows₩system32₩cmd.exe
-727379968
계속하려면 아무 키나 누르십시오 . . .
```



OverflowExample.java_오버플로우 (80)

- 결과값 산출할 때 Overflow 해결

```
1  public class OverflowExample {
2      public static void main(String[] args) {
3          /*
4          int x = 1000000;
5          int y = 1000000;
6          int z = x * y;
7          System.out.println(z);
8          */
9
10         long x = 1000000;
11         long y = 1000000;
12         long z = x * y;
13         System.out.println(z);
14     }
15 }
16
17
```

```
C:₩Windows₩system32₩cmd.exe
1000000000000
계속하려면 아무 키나 누르십시오 . . .
```

CheckOverflowExample.java_오버플로우 (81)

- 산술 연산 전에 Overflow 탐지

```
    |----+----1----+----2----+----3----+----4----+----5----+----6----+----7----+----8--
 1  public class CheckOverflowExample {
 2      public static void main(String[] args) {
 3          try {
 4              int result = safeAdd(2000000000, 2000000000);
 5              System.out.println(result);
 6          } catch(ArithmeticException e) {
 7              System.out.println("오버플로우가 발생하여 정확하게 계산할 수 없음");
 8          }
 9      }
10
11      public static int safeAdd(int left, int right)  {
12          if((right>0)) {
13              if(left>(Integer.MAX_VALUE - right)) {
14                  throw new ArithmeticException("오버플로우 발생");
15              }
16          } else {
17              if(left<(Integer.MIN_VALUE - right)) {
18                  throw new ArithmeticException("오버플로우 발생");
19              }
20          }
21          return left + right;
22      }
23  }
```

```
C:₩Windows₩system32₩cmd.exe
오버플로우가 발생하여 정확하게 계산할 수 없음
계속하려면 아무 키나 누르십시오 . . .
```

AccuracyExample1.java (82)

- 정확한 계산은 정수를 사용

```
    ├---+----1----+----2----+----3----+----4----+----5----+---
 1  public class AccuracyExample1 {
 2      public static void main(String[] args) {
 3          int apple = 1;
 4          double pieceUnit = 0.1;
 5          int number = 7;
 6
 7          double result = apple - number*pieceUnit;
 8
 9          System.out.println("사과 한개에서 ");
10          System.out.println("0.7 조각을 빼면, ");
11          System.out.println(result + " 조각이 남는다.");
12      }
13  }
14
15
```

```
C:₩Windows₩system32₩cmd.exe
사과 한개에서
0.7 조각을 빼면,
0.29999999999999993 조각이 남는다.
계속하려면 아무 키나 누르십시오 . . .
```

AccuracyExample2.java(82)

- 정확한 계산은 정수를 사용

```
    ├---+----1----+----2----+----3----+----4----+----5----+---
 1  public class AccuracyExample2 {
 2      public static void main(String[] args) {
 3          int apple = 1;
 4
 5          int totalPieces = apple * 10;
 6          int number = 7;
 7          int temp = totalPieces - number;
 8
 9          double result = temp/10.0;
10
11          System.out.println("사과 한개에서 ");
12          System.out.println("0.7 조각을 빼면, ");
13          System.out.println(result + " 조각이 남는다.");
14      }
15  }
16
```

```
C:₩Windows₩system32₩cmd.exe
사과 한개에서
0.7 조각을 빼면,
0.3 조각이 남는다.
계속하려면 아무 키나 누르십시오 . . .
```

InfinityAndNaNCheckExample.java(84)

- /와 %일 때의 주의 점 _NaN(Not a number)과 Infinity(무한대) 연산은 주의할 것

```
    |---+----1----+----2----+----3----+----4----+----5----+----
 1  public class InfinityAndNaNCheckExample {
 2      public static void main(String[] args) {
 3          int x = 5;
 4          double y = 0.0;
 5
 6          double z = x / y;
 7          //double z = x % y;
 8
 9          System.out.println(Double.isInfinite(z));
10          System.out.println(Double.isNaN(z));
11
12          //잘못된 코드
13          System.out.println(z + 2);
14
15          //알맞은 코드
16          if(Double.isInfinite(z) || Double.isNaN(z)) {
17              System.out.println("값 산출 불가");
18          } else {
19              System.out.println(z + 2);
20          }
21
22          //--
23
24          /*in
25          int
```

```
C:₩Windows₩system32₩cmd.exe
true
false
Infinity
값 산출 불가
계속하려면 아무 키나 누르십시오 . . .
```

InputDataCheckNaNExample1.java(85) ” NaN” 문자열의 문제점

- /와 %일 때의 주의 점 _NaN(Not a number)과 Infinity(무한대) 연산은 주의할 것

```
    |---+---1---+---2---+---3---+---4---+---5-
 1  public class InputDataCheckNaNExample1 {
 2      public static void main(String[] args) {
 3          String userInput = "NaN";
 4          double val = Double.valueOf( userInput );
 5
 6          double currentBalance = 10000.0;
 7
 8          currentBalance += val;
 9          System.out.println(currentBalance);
10      }
11  }
12
13
```

```
C:₩Windows₩system32₩cmd.exe
NaN
계속하려면 아무 키나 누르십시오 . . .
```

InputDataCheckNaNExample2.java(85) “NaN” 체크하고 연산 수행

- /와 %일 때의 주의 점 _NaN(Not a number)과 Infinity(무한대) 연산은 주의할 것

```
    |----+----1----+----2----+----3----+----4----+----5----+----6----+--
 1  public class InputDataCheckNaNExample2 {
 2      public static void main(String[] args) {
 3          String userInput = "NaN";
 4          double val = Double.valueOf(userInput);
 5
 6          double currentBalance = 10000.0;
 7
 8          if(Double.isNaN(val)) {
 9              System.out.println("NaN이 입력되어 처리할 수 없음");
10              val = 0.0;
11          }
12
13          currentBalance += val;
14          System.out.println(currentBalance);
15      }
16  }
17
```

```
C:₩Windows₩system32₩cmd.exe
NaN이 입력되어 처리할 수 없음
10000.0
계속하려면 아무 키나 누르십시오 . . .
```

4절. 이항 연산자

❖ 비교 연산자(==, !=, <, >, <=, >=) (87~91)

- 대소(<, <=, >, >=) 또는 동등(==, !=) 비교해 boolean 타입인 true/false 산출

구분	연산식			설명
동등 비교	피연산자	==	피연산자	두 피 연산자의 값이 같은지를 검사
	피연산자	!=	피연산자	두 피 연산자의 값이 다른지를 검사
크기 비교	피연산자	>	피연산자	피 연산자1이 큰지를 검사
	피연산자	>=	피연산자	피 연산자1이 크거나 같은지를 검사
	피연산자	<	피연산자	피 연산자1이 작은지를 검사
	피연산자	<=	피연산자	피 연산자1이 작거나 같은지를 검사

- 동등 비교 연산자는 모든 타입에 사용
- 크기 비교 연산자는 boolean 타입 제외한 모든 기본 타입에 사용
- 흐름 제어문인 조건문(if), 반복문(for, while)에서 주로 이용
  - 실행 흐름을 제어할 때 사용

CompareOperatorExample.java(89)_비교 연산자

- 대소(<, <=, >, >=) 또는 동등(==, !=) 비교해 boolean 타입인 true/false 산출

```java
package ch03.sec06;

public class CompareOperatorExample {
    public static void main(String[] args) {
        int num1 = 10;
        int num2 = 10;
        boolean result1 = (num1 == num2);
        boolean result2 = (num1 != num2);
        boolean result3 = (num1 <= num2);
        System.out.println("result1: " + result1);
        System.out.println("result2: " + result2);
        System.out.println("result3: " + result3);

        char char1 = 'A';
        char char2 = 'B';
        boolean result4 = (char1 < char2); //65 < 66
        System.out.println("result4: " + result4);

        int num3 = 1;
        double num4 = 1.0;
        boolean result5 = (num3 == num4);
        System.out.println("result5: " + result5);

        float num5 = 0.1f;
        double num6 = 0.1;
        boolean result6 = (num5 == num6);
        boolean result7 = (num5 == (float)num6);
        System.out.println("result6: " + result6);
        System.out.println("result7: " + result7);

        String str1 = "자바";
        String str2 = "Java";
        boolean result8 = (str1.equals(str2));
        boolean result9 = (! str1.equals(str2));
        System.out.println("result8: " + result8);
        System.out.println("result9: " + result9);
    }
}
```



StringEqualsExample.java(91)_문자열 비교

- 대소(<, <=, >, >=) 또는 동등(==, !=) 비교해 boolean 타입인 true/false 산출

```
    ├----+----1----+----2----+----3----+----4----+----5----
 1  public class StringEqualsExample {
 2      public static void main(String[] args) {
 3          String strVar1 = "신민철";
 4          String strVar2 = "신민철";
 5          String strVar3 = new String("신민철");
 6
 7          System.out.println( strVar1 == strVar2);
 8          System.out.println( strVar1 == strVar3);
 9          System.out.println();
10          System.out.println( strVar1.equals(strVar2));
11          System.out.println( strVar1.equals(strVar3));
12      }
13  }
14
```

```
C:₩Windows₩system32₩cmd.exe
true
false

true
true
계속하려면 아무 키나 누르십시오 . . .
```

4절. 이항 연산자

❖ 논리 연산자 (&&, ||, &, |, ^, !) (p.91~93)

- 논리곱(&&), 논리합(||), 배타적 논리합(^) ,논리 부정(!) 연산 수행
- 피연산자는 boolean 타입만 사용 가능

구분	연산식			결과	설명
AND (논리곱)	true	&& 또는 &	true	true	피 연사자 모두가 true 일 경우에만 연산 결과는 true
	true		false	false	
	false		true	false	
	false		false	false	
OR (논리합)	true	|| 또는 |	true	true	피 연산자 중 하나만 true 이면 연산 결과는 true
	true		false	true	
	false		true	ture	
	false		false	false	
XOR (배타적 논리합)	true	^	true	false	피 연산자가 하나는 ture 이고 다른 하나가 false 일 경우에만 연산 결과는 true
	true		false	true	
	false		true	ture	
	false		false	false	
NOT (논리부정)		!	true	false	피 연산자의 논리값을 바꿈
			false	true	

LogicalOperatorExample.java(92)_논리 연산자

```
    |----+----1----+----2----+----3----+----4----+----5----+----6----
 1  public class LogicalOperatorExample {
 2      public static void main(String[] args) {
 3          int charCode = 'A';
 4
 5          if( (charCode>=65) & (charCode<=90) ) {
 6              System.out.println("대문자 이군요");
 7          }
 8
 9          if( (charCode>=97) && (charCode<=122) ) {
10              System.out.println("소문자 이군요");
11          }
12
13          if( !(charCode<48) && !(charCode>57) ) {
14              System.out.println("0~9 숫자 이군요");
15          }
16
17          int value = 6;
18
19          if( (value%2==0) | (value%3==0) ) {
20              System.out.println("2 또는 3의 배수 이군요");
21          }
22
23          if( (value%2==0) || (value%3==0) ) {
24              System.out.println("2 또는 3의 배수 이군요");
25          }
26      }
27  }
28
```

```
C:₩Windows₩system32₩cmd.exe
대문자 이군요
2 또는 3의 배수 이군요
2 또는 3의 배수 이군요
계속하려면 아무 키나 누르십시오
```

4절. 이항 연산자

❖ 비트 연산자(&, |, ^, ~, <<, >>, >>>) (p.94~98)

- 비트(bit) 단위로 연산 하므로 0과 1이 피연산자
  - 0과 1로 표현이 가능한 정수 타입만 비트 연산 가능
  - 실수 타입인 float과 double은 비트 연산 불가
- 종류
  - 비트 논리 연산자(&, |, ^, ~)
  - 비트 이동 연산자(<<, >>, >>>)



4절. 이항 연산자

❖ 비트 논리 연산자(&, |, ^, ~)

- 피 연산자가 boolean타입일 경우 일반 논리 연산자
- 피연산자가 정수 타입일 경우 비트 논리 연산자로 사용
- 비트 연산자는 피연산자를 int타입으로 자동 타입 변환 후 연산 수행

구분	연산식			결과	설명
AND (논리곱)	1	&	1	1	두 비트가 모두가 1 일 경우에만 연산 결과는 1
	1		0	0	
	0		1	0	
	0		0	0	
OR (논리합)	1	|	1	1	두 비트 중 하나만 1 이면 연산 결과는 1
	1		0	1	
	0		1	1	
	0		0	0	
XOR (배타적 논리합)	1	^	1	0	두 비트 중 하나는 1 이고 다른 하나가 0 일 경우 연산 결과는 1
	1		0	1	
	0		1	1	
	0		0	0	
NOT (논리부정)		~	1	0	보수
			0	1	

BitLogicalExample.java(96)_비트 논리 연산자

```java
package ch03.sec08;

public class BitLogicExample {
    public static void main(String[] args) {
        System.out.println("45 & 25 = " + (45 & 25));
        System.out.println("45 | 25 = " + (45 | 25));
        System.out.println("45 ^ 25 = " + (45 ^ 25));
        System.out.println("~45 = " + (~45));
        System.out.println("-------------------------------");

        byte receiveData = -120;

        //방법1: 비트 논리곱 연산으로 Unsigned 정수 얻기
        int unsignedInt1 = receiveData & 255;
        System.out.println(unsignedInt1);

        //방법2: 자바 API를 이용해서 Unsigned 정수 얻기
        int unsignedInt2 = Byte.toUnsignedInt(receiveData);
        System.out.println(unsignedInt2);

        int test = 136;
        byte btest = (byte) test;
        System.out.println(btest);
    }
}
```

4절. 이항 연산자

❖ 비트 이동 연산자(<<, >>, >>>)

- 정수 데이터의 비트를 좌측 또는 우측으로 밀어 이동시키는 연산 수행

구분	연산식			설명
이동 (쉬프트)	a	<<	b	정수 a 의 각 비트를 b 만큼 왼쪽으로 이동 (빈자리는 0 으로 채워진다.)
	a	>>	b	정수 a 의 각 비트를 b 만큼 오른쪽으로 이동 (빈자리는 정수 a 의 최상위 부호 비트(MSB)와 같은 값으로 채워진다.)
	a	>>>	b	정수 a 의 각 비트를 오른쪽으로 이동 (빈자리는 0 으로 채워진다.)



BitShiftExample.java(98)_비트 이동 연산자

```java
package ch03.sec09;

public class BitShiftExample1 {
    public static void main(String[] args) {
        int num1 = 1;
        int result1 = num1 << 3;
        int result2 = num1 * (int) Math.pow(2, 3);
        System.out.println("result1: " + result1);
        System.out.println("result2: " + result2);

        int num2 = -8;
        int result3 = num2 >> 3;
        int result4 = num2 / (int) Math.pow(2, 3);
        System.out.println("result3: " + result3);
        System.out.println("result4: " + result4);
    }
}
```



4절. 이항 연산자

- **대입 연산자(=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=)**
  - 오른쪽 피연산자의 값을 좌측 피연산자인 변수에 저장
  - 모든 연산자들 중 가장 낮은 연산 순위 -> 제일 마지막에 수행
  - 종류
    - 단순 대입 연산자
    - 복합 대입 연산자
      - 정해진 연산을 수행한 후 결과를 변수에 저장



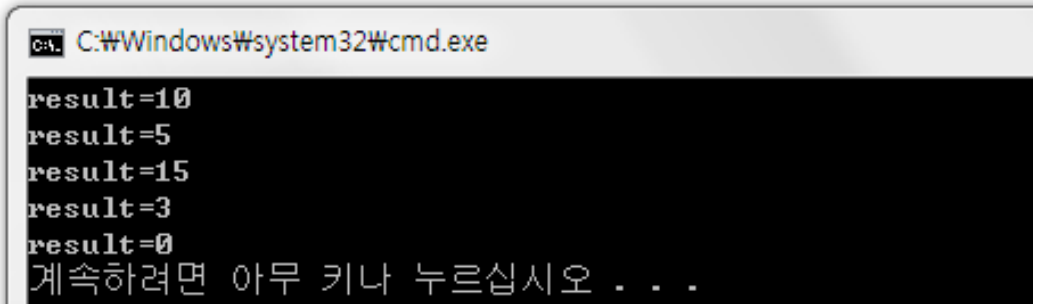
4절. 이항 연산자

❖ 대입 연산자의 종류

구분	연산식			설명
단순 대입 연산자	변수	=	피연산자	우측의 피연산자의 값을 변수에 저장
복합 대입 연산자	변수	+=	피연산자	우측의 피연산자의 값을 변수의 값과 더한 후에 다시 변수에 저장 (변수=변수+피연산자 와 동일)
	변수	-=	피연산자	우측의 피연산자의 값을 변수의 값에서 뺀 후에 다시 변수에 저장 (변수=변수-피연산자 와 동일)
	변수	*=	피연산자	우측의 피연산자의 값을 변수의 값과 곱한 후에 다시 변수에 저장 (변수=변수*피연산자 와 동일)
	변수	/=	피연산자	우측의 피연산자의 값으로 변수의 값을 나눈 후에 다시 변수에 저장 (변수=변수/피연산자 와 동일)
	변수	%=	피연산자	우측의 피연산자의 값으로 변수의 값을 나눈 후에 나머지를 변수에 저장 (변수=변수%피연산자 와 동일)
	변수	&=	피연산자	우측의 피연산자의 값과 변수의 값을 & 연산 후 결과를 변수에 저장 (변수=변수&피연산자 와 동일)
	변수	|=	피연산자	우측의 피연산자의 값과 변수의 값을 | 연산 후 결과를 변수에 저장 (변수=변수|피연산자 와 동일)
	변수	^=	피연산자	우측의 피연산자의 값과 변수의 값을 ^ 연산 후 결과를 변수에 저장 (변수=변수^피연산자 와 동일)
	변수	<<=	피연산자	우측의 피연산자의 값과 변수의 값을 << 연산 후 결과를 변수에 저장 (변수=변수<<피연산자 와 동일)
	변수	>>=	피연산자	우측의 피연산자의 값과 변수의 값을 >> 연산 후 결과를 변수에 저장 (변수=변수>>피연산자 와 동일)
	변수	>>>=	피연산자	우측의 피연산자의 값과 변수의 값을 >>> 연산 후 결과를 변수에 저장 (변수=변수>>>피연산자 와 동일)

AssignmentOperatorExample.java(100)_대입 연산자

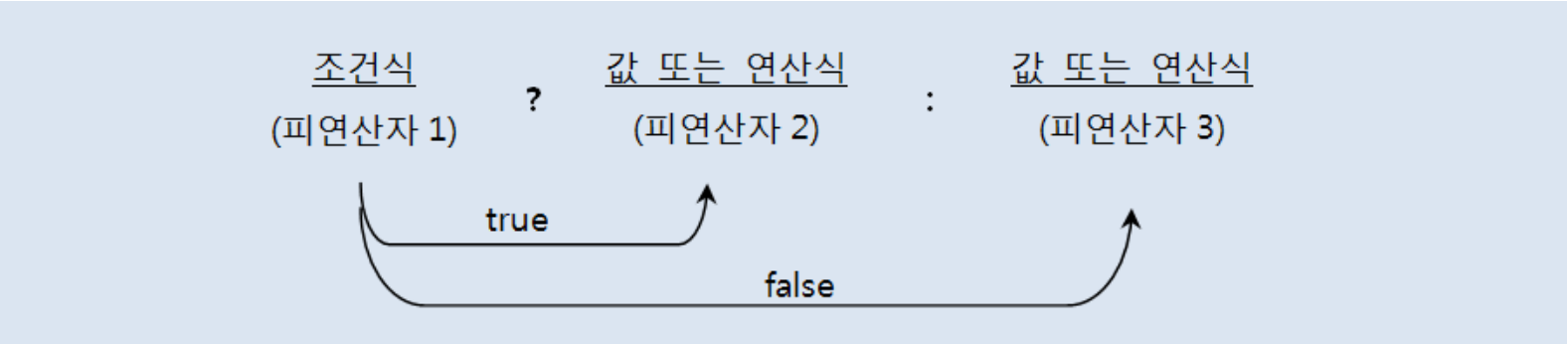
```java
public class AssignmentOperatorExample {
    public static void main(String[] args) {
        int result = 0;
        result += 10;
        System.out.println("result=" + result);
        result -= 5;
        System.out.println("result=" + result);
        result *= 3;
        System.out.println("result=" + result);
        result /= 5;
        System.out.println("result=" + result);
        result %= 3;
        System.out.println("result=" + result);
    }
}
```



5절. 삼항 연산자

❖ 삼항 연산자란?

- 세 개의 피연산자를 필요로 하는 연산자
- 앞의 조건식 결과에 따라 콜론 앞 뒤의 피연산자 선택 -> 조건 연산식




```
int score = 95;
char grade = (score>90) ?  'A'  :  'B'
```

=

```
int score = 95;
char grade;
if(score>90) {
   grade = 'A';
} else {
   grade = 'B';
}
```

ConditionalOperationExample.java(100)_삼항 연산자

```java
public class ConditionalOperationExample {
    public static void main(String[] args) {
        int score = 85;
        char grade = (score > 90) ? 'A' : ( (score > 80) ? 'B' : 'C' );
        System.out.println(score + "점은 " + grade + "등급입니다.");
    }
}
```

